EE5203 Lab 6 Guide

Timer interrupts: a 1 ms tick instead of `HAL_Delay()` - Basri Erdogan

Mission: Build a 1 ms tick from a TIM2 update interrupt, drive a heartbeat blink from it with no `HAL_Delay()` anywhere, and prove in the SFR view that the hardware counts while your loop stays free.

This lab uses Lecture 7 (timer time base, `PSC`/`ARR`, update interrupt, millisecond tick, bare-metal timer registers).

Prepare before class

- ☐ Bring your board and a USB **data** cable.
- ☐ Re-read the theory slides up to "A 1 ms tick counter from the timer."
- ☐ Compute from 16 MHz: which `PSC`/`ARR` give a 1 ms update event?
- ☐ Know which register bit records the update event and who clears it.

Learning objectives

By checkoff time, you can:

1. configure a TIM2 time base in CubeMX from the real 16 MHz clock;
2. run an update-interrupt callback and keep it short;
3. schedule LED phases from a millisecond counter, wrap-safe;
4. drive a bare-metal microsecond timer and explain every register write.

Hardware and files

Item	Value
Board	Nucleo-F401RE: LED on PA5, **no wiring**
Clock	**HSI, PLL off** (you set this in CubeMX, step 2 below): `SYSCLK` = `PCLK1` = timer clock = **16 MHz**
Timer	TIM2, internal clock source
Tools	STM32CubeMX (new project) + VS Code
Project name	`EE5203_L06_<studentnumber>`
File you edit	`main.c`, only inside the `USER CODE` regions

Configure in CubeMX

Start of the session: if `C:\Work\<studentnumber>` does not contain your Lab 5 project, copy **all** your project folders from `H:\EE_Labs` into it first.

1. **STM32CubeMX:** Board Selector → **NUCLEO-F401RE** → accept the default initialization.
2. **Clock Configuration tab** (next to Pinout): set **System Clock Mux** to **HSI** and the **AHB**, **APB1** and **APB2** prescalers to **/1**. Every box on the right now reads **16**, including **APB1 Timer clocks (MHz)**. The board default runs the chip at 84 MHz through the PLL; with that setting every time in this lab is 5.25 times too short.
3. **PA5** as in Lab 4: **Reset_State**, then `GPIO_Output` (defaults: push-pull, no pull, low speed).
4. **Timers → TIM2**: Clock Source = `Internal Clock`; Prescaler `15` (16 MHz ÷ 16 = 1 MHz), Counter Period `999` (1 MHz ÷ 1000 → 1 ms).
5. **System Core → NVIC**: enable **TIM2 global interrupt**.
6. Project Manager: name `EE5203_L06_<studentnumber>`, location `C:\Work\<studentnumber>`, Toolchain/IDE = **CMake** → Generate Code.
7. **Copy `C:\Users\Public\EE5203_template\.vscode` into the project folder**, as in every lab. Without it the build stops with `...FLASH.ld: syntax error`. (Own laptop: skip this step and use **Set Up STM32Cube project(s)**, as in Lab 5.)
8. **Verify, do not add:** `MX_TIM2_Init()` in `main.c`, and `TIM2_IRQHandler()` calling `HAL_TIM_IRQHandler(&htim2)` in `stm32f4xx_it.c`. CubeMX wrote both.

Checkpoint A - The tick runs

Three additions in `main.c`, each in its own `USER CODE` region:

1. **In `USER CODE BEGIN PV`**, the millisecond counter:

```
volatile uint32_t g_ms = 0;
```

2. **In `USER CODE BEGIN 4`** (near the end of `main.c`), the callback. The HAL calls it at every update event:

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2) {
        g_ms++;
```

```
    }
}
```

3. **In `USER CODE BEGIN 2`** (before `while (1)`), start the timer in interrupt mode:

```
HAL_TIM_Base_Start_IT(&htim2);
```

`MX_TIM2_Init()` only writes `PSC` and `ARR`. Without this line `CNT` stays at 0 and the callback never runs.

4. Build (**Ctrl+Shift+B**), start the debugger (**F5**), put a breakpoint on `g_ms++`, Continue.

Evidence for Checkpoint A: the breakpoint in the callback is hit; `g_ms` grows between pauses; in the SFR view `TIM2->PSC` = 15, `TIM2->ARR` = 999, and `TIM2->CNT` is moving.

Checkpoint B - Heartbeat without delay

Blink a heartbeat, LD2 **on for 100 ms, off for 900 ms**, timed only from `g_ms`. Two additions in `main.c`:

1. **In `USER CODE BEGIN 2`**, below `HAL_TIM_Base_Start_IT(&htim2);`, the two state variables:

```
uint32_t t_phase = 0;        /* g_ms value at the last LED change */
uint8_t  led_on  = 0;
```

2. **In `USER CODE BEGIN 3`** (inside `while (1)`), the scheduler. Nothing here waits; the loop stays free for other work:

```
uint32_t wait = led_on ? 100U : 900U;

if ((g_ms - t_phase) >= wait) {
    t_phase = g_ms;
    led_on = !led_on;
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5,
                      led_on ? GPIO_PIN_SET : GPIO_PIN_RESET);
}
```

> New syntax: `condition ? a : b` (the conditional operator)
>
> `wait = led_on ? 100U : 900U;` means: if `led_on` is true, `wait` gets `100U`, otherwise `900U`. It is the same as
>
> ```
> if (led_on) { wait = 100U; } else { wait = 900U; }
> ```
>
> Read `?` as "then" and `:` as "else". The `if` form is also accepted.

The unsigned subtraction stays correct when `g_ms` wraps: Lab 5's debounce arithmetic again.

Evidence for Checkpoint B: the 100/900 pattern is visible and `main.c` contains no `HAL_Delay()` call.

Checkpoint C - Bare-metal microsecond timer

Now program TIM2 with register writes only, as a free-running **microsecond clock**, and blink LD2 **250 ms on / 250 ms off** from it with **no HAL timer calls**. TIM2 cannot be the 1 ms tick and the microsecond clock at the same time, so this checkpoint switches the HAL tick off. Three changes in `main.c`:

1. **In `USER CODE BEGIN 0`** (above `main()`), the init and the wait function:

```
void tim2_us_init(void)
{
    RCC->APB1ENR |= (1U << 0);    /* TIM2 clock enable: APB1, not AHB1  */
    TIM2->PSC = 15;               /* 16 MHz / 16 = 1 MHz: 1 count = 1 us */
    TIM2->ARR = 0xFFFFFFFFU;      /* free-running: TIM2 is 32-bit        */
    TIM2->EGR = (1U << 0);        /* UG: load PSC now                    */
    TIM2->CR1 |= (1U << 0);       /* CEN: start counting                 */
}

void delay_us(uint32_t us)
{
    uint32_t start = TIM2->CNT;
    while ((TIM2->CNT - start) < us) { }   /* wait for us counts */
}
```

2. **In `USER CODE BEGIN 2`**, replace the HAL start with your init. Keep the old line as a comment; you switch back for the extension:

```
/* HAL_TIM_Base_Start_IT(&htim2); */   /* off: Checkpoint C owns TIM2 */
tim2_us_init();
```

3. **In `USER CODE BEGIN 3`**, switch the Checkpoint B scheduler off and add the blink. `#if 0` … `#endif` makes the compiler skip the lines between them, so your Checkpoint B code stays in the file for the checkoff:

```
#if 0
    /* your Checkpoint B lines stay here, unchanged */
#endif
HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
delay_us(250000U);                    /* 250 000 us = 250 ms */
```

   `delay_us()` waits like `HAL_Delay()`, but it counts on `TIM2->CNT`, not on the SysTick interrupt. `g_ms` no longer grows here: the update interrupt is off.

Build, flash, Continue.

Evidence for Checkpoint C: the 250/250 blink runs; in the SFR view `TIM2->ARR` = `0xFFFFFFFF` and `CNT` is moving; you can explain the `EGR` write and why the subtraction survives counter wrap (~71 minutes at 1 MHz).

Individual extension - Button-selected speed

Switch back to the HAL tick first: restore `HAL_TIM_Base_Start_IT(&htim2);`, delete `tim2_us_init();`, delete the two blink lines and the `#if 0` / `#endif` pair. Then, in the Checkpoint B loop: B1 (PC13, input, **no pull**; the external resistor is on the board) cycles the blink period through 500 → 200 → 50 ms. Debounce with 50 ms of `g_ms`, exactly as in Lab 5. Optional: add `__WFI();` at the loop end and confirm timing still holds.

Acceptance checklist

- ☐ `.ioc`: TIM2 PSC 15 / ARR 999, NVIC enabled; PA5 output.
- ☐ Callback checks `htim->Instance` and only increments a counter.
- ☐ SFR evidence: PSC, ARR, moving CNT shown for Checkpoint A.
- ☐ No `HAL_Delay()` anywhere in the HAL tasks.
- ☐ Bare-metal version explained register by register.
- ☐ Extension demonstrated with clean speed switching.

Individual oral check

- Why 15 and 999, and what breaks without the two +1s?
- What would happen if the callback did the LED writes and took 2 ms?
- Why is `g_ms` declared `volatile`, and why is the subtraction wrap-safe?
- `TIM2` is on which bus, and which register enables its clock?

Submit

Submit `main.c` as it is at the end of the session (the Checkpoint C functions stay in `USER CODE BEGIN 0`), and one SFR-view screenshot showing `TIM2->PSC`, `TIM2->ARR`, and a nonzero `TIM2->CNT`.

If you are stuck

Walk the pipeline left to right and fix the **first** silent stage: `PSC`/`ARR` values in the SFR view → `CNT` moving → `UIF` setting in `SR` → NVIC enabled → breakpoint in the callback → `g_ms` growing → your loop condition. **`...FLASH.ld: syntax error` at build?** `.vscode` is missing from the project folder (step 7). **CNT frozen?** Timer never started or clock not enabled. **Callback never runs?** NVIC checkbox or `Start_IT` missing. **Everything about 5 times too fast** (the heartbeat, the tick)? The Clock Configuration tab is still on the PLL at 84 MHz: System Clock Mux = HSI, prescalers /1, generate code again. **Pattern timing drifts?** You reset `t_phase` to `g_ms` somewhere other than the phase change. **Checkpoint C blink never changes?** `tim2_us_init()` is not called before the loop, so `CNT` is frozen and `delay_us()` never returns.